AI Software Engineering Fundamentals

Independent study notes for an AI Software Engineering bootcamp. These are original summaries, not official course materials.

1. Git and GitHub

Git is a distributed version control system that records changes in a repository. GitHub is a hosting and collaboration platform built around Git repositories. A commit captures a snapshot of tracked changes; a branch allows isolated development; a pull request proposes changes for review before merging.

A practical workflow is: create a feature branch, implement a small change, run tests, commit with a meaningful message, push, open a pull request, review the diff, and merge after checks pass. Avoid committing API keys, passwords, or environment files.

2. Engineering principles

Separation of concerns keeps UI, business rules, and persistence logic in distinct modules. High cohesion means each module has a focused responsibility; low coupling reduces unnecessary dependencies. DRY encourages removing harmful duplication, while KISS encourages choosing the simplest design that meets requirements.

Automated tests verify behavior at different levels. Unit tests target isolated functions; integration tests check interactions such as database queries; end-to-end tests exercise a user workflow. Code review checks correctness, readability, security, and maintainability.

3. Software architecture

A layered application may have a presentation layer, application or service layer, domain logic, and data access layer. The UI should not directly own database credentials. Dependency inversion lets business logic depend on interfaces rather than concrete services.

For a task management app, a TaskService might validate task rules while a TaskRepository handles database reads and writes. A REST API exposes operations through endpoints such as GET /tasks and POST /tasks. Authentication establishes identity; authorization checks whether that identity may access a resource.

4. Prompt engineering

A useful prompt states the task, relevant context, constraints, expected output format, and criteria for success. Examples can demonstrate the desired pattern. For complex coding tasks, provide the specific files, errors, and acceptance criteria rather than the entire repository.

Prompt injection occurs when untrusted content tries to override higher-priority instructions. Treat retrieved documents and user-supplied files as data, not as authority. Verify model output with tests and trusted references, especially for security-sensitive code.

5. Clean code and refactoring

Choose descriptive names, keep functions focused, make error handling explicit, and avoid unnecessary abstractions. Refactoring changes internal structure without intentionally changing external behavior. Tests should pass before and after refactoring.

Technical debt is the future cost of shortcuts. Not every shortcut is bad: document intentional tradeoffs and revisit them when requirements or scale change. Observability uses logs, metrics, and traces to diagnose production behavior.

6. Practical example

Suppose a Next.js task app stores tasks in MySQL. A server action validates the input, obtains the authenticated user ID, and calls a repository that writes only rows owned by that user. An integration test checks ownership filtering; an end-to-end test checks task creation in the browser.